

FOODRESCUE

Intelligent Surplus Food Redistribution
PROJECT REPORT

Submitted by

Name: Akhandam Sri Sai Venkata Karthikeya

Registration Number: 25BAI10737

Course: CSE -2006 Programming in Java

Institution: VIT BHOPAL

Vityarthi Project Submission

Table of Contents

1. Introduction

2. Problem Statement

3. Functional Requirements

4. Non-Functional Requirements

5. Workflow / Process Flow

6. diagrams

a. Use Case Design

b. Class / Component Design

c. Sequence Flow

7. Design Decisions and Rationale

8. Implementation Details

9. Screenshots and Results

10. Testing Approach

11. Challenges Faced

1. Introduction

Food waste is a major practical problem when usable surplus food is discarded while individuals and organizations continue to experience food shortages. Restaurants, hotels, catering services and events may generate surplus edible food that can potentially be redistributed if it is identified and allocated before expiry.

FoodRescue is a Java-based command-line application developed to manage surplus food donations and food requests. The system records donor and receiver details, tracks food quantity and expiry time, manages pending requests, identifies suitable matches, creates allocations, identifies urgent food and maintains allocation history. The project is designed with a modular structure using Model and Service packages and can be compiled and executed directly through the terminal.

2. Problem Statement

Large quantities of edible surplus food from restaurants, hotels, catering services and events are wasted every day. At the same time, NGOs, shelters and relief organizations may experience food shortages. When multiple food donations and food requests exist simultaneously, it becomes difficult to determine which food should be allocated first and which receiver should receive it. Factors such as food expiry time, available quantity, receiver demand and emergency level must be considered. Manual coordination can cause delays, resulting in food being wasted while genuine food requirements remain unmet.

Proposed Solution

FoodRescue provides a Java-based command-line solution for managing surplus food donations and food requests. It tracks available food, monitors expiry time, records receiver requirements and supports intelligent matching and allocation. The system also provides urgent-food monitoring, statistics and

file-based allocation history.

3. Functional Requirements

<i>Module / Feature</i>	<i>Description</i>
<i>Food Donation Management</i>	<i>Capture donor information, food name, type, quantity and expiry time; add food to the available collection.</i>
<i>Food Request Management</i>	<i>Capture receiver information, required food type, quantity and emergency level; create a pending request.</i>
<i>Food Viewing</i>	<i>Display available food records for monitoring and selection.</i>
<i>Request Viewing</i>	<i>Display pending food requests.</i>
<i>Smart Matching and Allocation</i>	<i>Search available food and pending requests for a suitable match and create an allocation.</i>
<i>Allocation History</i>	<i>Display completed allocations maintained by the matching service.</i>
<i>Urgent Food Monitoring</i>	<i>Identify available, non-expired food items nearing expiry.</i>
<i>System Statistics</i>	<i>Show counts such as available food, pending requests and completed allocations.</i>
<i>File Handling</i>	<i>Save allocation history to a text file and read the saved allocation history.</i>
<i>Input Validation</i>	<i>Reject empty string input and handle invalid numeric input.</i>

4. Non-Functional Requirements

Usability

The menu-driven interface provides numbered options and clear prompts for user interaction.

Reliability
Food and request status are managed through dedicated service classes, and invalid input is handled to reduce incorrect processing.

Maintainability

The application separates domain entities into the Model package and business operations into the Service package.

Performance

The project is designed for small to moderate console-based datasets and performs direct collection-based processing.

Error Handling

Numeric input is validated using exception handling, while empty text input and invalid emergency levels are rejected.

Portability

5. System Architecture

FoodRescue follows a simple layered organization. The user interacts with Main.java through the command-line menu. Main coordinates the service layer. Service classes manage operations on model objects, while FileService provides persistent text-file storage for allocation history.

USER / COMMAND LINE

Main.java

Service Layer: FoodService | RequestService | MatchingService |

FileService

Model Layer: Donor | Receiver | FoodItem | FoodRequest |

Allocation

Storage: allocations.txt

Architecture Explanation

Main.java controls program flow and user interaction. FoodService manages food items, RequestService manages food requests, MatchingService performs matching and maintains allocations, and FileService saves and reads allocation records. Model classes

represent the main entities used by the application.



6. Design

START

Display Main Menu

User selects an operation

Add / View / Match / Monitor / Save

Process through Service classes

Display result

Return to menu or Exit

The application repeatedly displays the main menu until the user selects Exit. Input is validated before creating domain objects.

Matching is initiated from the menu, and the resulting allocation is displayed and recorded by the matching component.

a. Use Case Design

Primary actors are represented by the people interacting with the system: donors provide surplus food, receivers submit requirements, and a system operator/coordinator uses the command-line application to manage records and allocations.

<i>Actor</i>	<i>Main Use Cases</i>
<i>Donor / Coordinator</i>	<i>Add Food Donation, View Available Food, View Urgent Food</i>
<i>Receiver / Coordinator</i>	<i>Add Food Request, View Pending Requests</i>
<i>Coordinator</i>	<i>Find Best Match, Allocate Food, View Allocation History, View Statistics</i>
<i>Coordinator</i>	<i>Save Allocation History, View Saved Allocation History</i>

```
===== MAIN MENU =====
1. Add Food Donation
2. Add Food Request
3. View Available Food
4. View Pending Requests
5. Find Best Match and Allocate Food
6. View Allocation History
7. View Urgent Food
8. View System Statistics
9. Save Allocation History to File
10. View Saved Allocation History
0. Exit
```

b. Class / Component Design

Model Package

Donor stores donor-related information. Receiver stores receiver information and emergency level. FoodItem represents donated food with quantity, expiry time, donor reference and allocation/expiry status. FoodRequest represents a receiver's required food type and quantity with pending status. Allocation represents a successful relationship between food and a request.

Service Package

FoodService manages food collections and provides operations such as adding, viewing and identifying urgent food. RequestService manages food requests and pending requests. MatchingService performs the best-match operation and maintains allocation history. FileService writes and reads allocation records using file handling.

Main.java

Main.java provides the command-line menu, collects user input, validates basic input and coordinates calls to the service classes.

```
Enter your choice: 2

===== ADD FOOD REQUEST =====
Enter receiver name: Hope NGO
Enter organization name: Hope Foundation
Enter phone number: 9123456789
Enter email: hope@gmail.com
Enter address: Vellore
Emergency Levels: 1 = LOW, 2 = MEDIUM, 3 = HIGH
Enter emergency level: 3
Enter food type required: Vegetarian
Enter quantity required: 40
```

c. Sequence Flow: Food Matching

- 1. The user selects the matching option from the main menu.*
- 2. Main.java requests the food collection from FoodService and request collection from RequestService.*
- 3. Main.java calls MatchingService to find a suitable match.*
- 4. MatchingService evaluates the available records and creates an Allocation when a suitable match is found.*
- 5. The allocation result is returned to Main.java and displayed to the user.*
- 6. Allocation history can later be viewed or saved through FileService.*

```
===== SMART FOOD MATCHING =====  
  
BEST MATCH FOUND SUCCESSFULLY!
```

7. Design Decisions and Rationale

Java

Java was selected because it supports object-oriented programming, collections, exception handling, file handling and command-line execution.

Object-Oriented Design

Separate classes represent donors, receivers, food items, requests and allocations, making the domain easier to model.

Model-Service Separation

Keeping entities and operational logic in separate packages improves readability and maintainability. Collections

Collections allow multiple food items, requests and allocations to be managed while the program is running.

Java Time API

LocalDateTime is used to represent expiry time and support time-based food monitoring. File Handling

Text-file storage provides a lightweight way to persist allocation history without introducing a database dependency.

Input Validation

Validation loops and NumberFormatException handling provide a safer command-line experience.

8. Implementation Details

The application starts from Main.java. Static service instances are created for food, requests, matching and file operations. The program runs inside a loop that displays a numbered menu. User input is collected through Scanner helper methods.

getIntegerInput() handles invalid numeric input using

NumberFormatException, while getStringInput() prevents empty text values.

When adding a donation, donor information and food details are collected and a FoodItem is created with an expiry time calculated using `LocalDateTime.now().plusHours(expiryHours)`. When adding a request, receiver information and an emergency level from 1 to 3 are collected before a FoodRequest is created. Matching uses the collections managed by the food and request services.

Project Structure



9. Screenshots and Results

Insert screenshots captured from the actual FoodRescue terminal execution. Screenshots should demonstrate that the submitted source code works correctly. Recommended screenshots are listed below.

```
===== MAIN MENU =====
1. Add Food Donation
2. Add Food Request
3. View Available Food
4. View Pending Requests
5. Find Best Match and Allocate Food
6. View Allocation History
7. View Urgent Food
8. View System Statistics
0. Exit
=====
Enter your choice: 1
```

Food donation added successfully.

Enter your choice: 1

```
===== ADD FOOD DONATION =====
Enter donor name: ABC Restaurant
Enter organization name: ABC Restaurant
Enter phone number: 9876543210
Enter email: abc@gmail.com
Enter address: Vellore
Enter food name: Vegetable Meals
Enter food type (Meals/Vegetarian/etc): Vegetarian
Enter quantity: 50
Enter hours until expiry: 5
```

Enter your choice: 2

```
===== ADD FOOD REQUEST =====
Enter receiver name: Hope NGO
Enter organization name: Hope Foundation
Enter phone number: 9123456789
Enter email: hope@gmail.com
Enter address: Vellore
Emergency Levels: 1 = LOW, 2 = MEDIUM, 3 = HIGH
Enter emergency level: 3
Enter food type required: Vegetarian
Enter quantity required: 40
```

Food request added successfully.

Enter your choice: 3

```
===== AVAILABLE FOOD =====

Food ID: 1
Food Name: Vegetable Meals
Food Type: Vegetarian
Quantity: 50
Expiry Time: 2026-09-15T05:00
Donor: ABC Restaurant
Status: Available
```

Enter your choice: 4

===== PENDING REQUESTS =====

Request ID: 1
Receiver: Hope NGO
Organization: Hope Foundation
Food Type Required: Vegetarian
Quantity Required: 40
Emergency Level: HIGH
Status: Pending

Enter your choice: 5

===== SMART FOOD MATCHING =====

BEST MATCH FOUND SUCCESSFULLY!

Enter your choice: 6

===== ALLOCATION HISTORY =====

Allocation ID: 1

Food: Vegetable Meals
Receiver: Hope NGO
Quantity Allocated: 40

Status: Completed

Enter your choice: 7

===== URGENT FOOD ITEMS =====

Food ID: 2
Food Name: Bread
Food Type: Bakery
Quantity: 20
Hours Until Expiry: 3

Urgent food item requires allocation.

Enter your choice: 9

Allocation history saved successfully.

Enter your choice: 10

```
===== SAVED ALLOCATION HISTORY =====  
  
Allocation ID: 1  
Food: Vegetable Meals  
Receiver: Hope NGO  
Quantity Allocated: 40  
Status: Completed
```

10. Testing Approach

<i>Test Case</i>	<i>Expected Result</i>
<i>Valid food donation</i>	<i>Food item is added and becomes available for viewing.</i>
<i>Valid food request</i>	<i>Request is added and appears as pending.</i>
<i>Invalid numeric input</i>	<i>Program displays an invalid-input message and asks again.</i>
<i>Invalid emergency level</i>	<i>Program accepts only levels 1, 2 or 3.</i>
<i>Matching with suitable data</i>	<i>A best-match allocation is created and displayed.</i>
<i>Matching without suitable data</i>	<i>Program reports that no suitable match was found.</i>
<i>Urgent food check</i>	<i>Eligible food nearing expiry is identified.</i>
<i>Save allocation history</i>	<i>Allocation data is written to the storage file.</i>
<i>View saved history</i>	<i>Previously saved allocation data is displayed.</i>

11. Challenges Faced

- *Designing a logical workflow connecting food donations, food requests and allocations.*
- *Managing food expiry information and identifying urgent food.*
- *Organizing the project into modular Model and Service packages.*
- *Handling invalid command-line input without terminating the program.*
- *Maintaining allocation history and implementing file-based saving and viewing.*
- *Ensuring the project can compile and run directly through the terminal.*
- *Preparing the repository with README.md, STATEMENT.md and .gitignore for GitHub submission.*

12. Learnings and Key Takeaways

- *Applying object-oriented programming to a real-world problem domain.*
- *Creating and using multiple classes with clear responsibilities.*
- *Using Java Collections to manage application data.*
- *Using LocalDateTime for expiry-related processing.*
- *Handling invalid input using exception handling and validation.*
- *Using file handling for simple persistent storage.*
- *Structuring a Java application into packages.*
- *Compiling and running a Java application through the terminal.*
- *Using Git and GitHub to organize and submit source code.*

13. Future Enhancements

- *Integrate a MySQL database using JDBC for permanent structured storage.*
- *Add user authentication and role-based access.*
- *Develop a graphical or web-based user interface.*
- *Add location-based donor and receiver matching.*
- *Introduce notifications for food nearing expiry and high-priority requests.*
- *Add reporting dashboards and more detailed analytics.*
- *Support partial allocations and more advanced optimization strategies.*

14. References

- 1. Java Documentation, Oracle / OpenJDK documentation — Java language, collections, exception handling, file handling and date-time APIs.*
- 2. Course materials and concepts related to Java programming and object-oriented programming.*
- 3. GitHub documentation for repository and version-control workflow.*